High Performance Computing

Assignment 08 – Parallel Interpolation with Particle Mover using MPI+OpenMP

Tirth Kaushal Gandhi – 202301413
Dhyey Patel – 202301415
Course: High Performance Computing (HPC)
Instructor: Prof. Bhaskar Chaudhury

April 2026

Abstract

This report presents a hybrid **MPI + OpenMP** parallel implementation of the bilinear scatter-point-to-mesh interpolation algorithm coupled with a reverse-interpolation particle mover (PIC pipeline). The implementation uses **particle decomposition** at the MPI level: Rank 0 reads particles once and distributes slices to all ranks via `MPI_Scatterv`. Each rank performs thread-private interpolation (eliminating race conditions without atomic updates), followed by an `MPI_Reduce` to assemble the global mesh, normalisation, broadcast, and a parallel mover step. Updated particles are collected via `MPI_Gatherv` and redistributed for the next iteration.

1 Hardware Architecture

Table 1: Cluster node specifications (`gics1`–`gics4`)

Component	Specification
Nodes used	`gics1`, `gics2`, `gics3`, `gics4`
CPU Model	Intel Xeon E5-2640 v3
Base Clock	2.60 GHz
Physical Cores/Node	16 (2 sockets × 8 cores)
Logical CPUs/Node	16
Total logical CPUs	64 (4 nodes × 16)
L3 Cache/Socket	20 MB shared
MPI Library	OpenMPI 1.8.8
Compiler	GCC with `-O3 -fopenmp -march=native`

The MPI–OpenMP layout used across the core sweep is shown in Table 2. The strategy keeps MPI ranks at 4 (one per node) and scales OpenMP threads per rank to fill available cores.

Table 2: MPI rank and OpenMP thread layout for the core sweep

Total Cores	MPI Ranks	OMP Threads / Rank
2	2	1
4	4	1
8	4	2
16	4	4
32	4	8
64	4	16

2 Problem Description

Given N scattered particles $\{(x_i, y_i)\}$ in the unit square $[0,1]^2$ with $f_i = 1$, values are interpolated onto a structured $(N_x + 1) \times (N_y + 1)$ mesh with uniform spacing $\Delta x = 1/N_x$, $\Delta y = 1/N_y$.

Bilinear (CIC) Interpolation Weights

For particle (x_i, y_i) in cell (g_x, g_y) where $g_x = \lfloor x_i/\Delta x \rfloor$:

$$l_x = x_i - g_x \Delta x, \qquad l_y = y_i - g_y \Delta y$$

$$a_1 = (\Delta x - l_x)(\Delta y - l_y) \to (g_x,\ g_y), \qquad a_2 = l_x(\Delta y - l_y) \to (g_x{+}1,\ g_y)$$

$$a_3 = (\Delta x - l_x)\, l_y \to (g_x,\ g_y{+}1), \qquad a_4 = l_x\, l_y \to (g_x{+}1,\ g_y{+}1)$$

Mover (Reverse Interpolation)

The same CIC weights are used to gather a field value F_i from the mesh for each particle:

$$F_i = a_1 \cdot F[g_x, g_y] + a_2 \cdot F[g_x{+}1, g_y] + a_3 \cdot F[g_x, g_y{+}1] + a_4 \cdot F[g_x{+}1, g_y{+}1]$$

The particle position is then updated as:

$$x_i^{\text{new}} = x_i + F_i \cdot \Delta x, \qquad y_i^{\text{new}} = y_i + F_i \cdot \Delta y$$

Particles leaving $[0,1]^2$ are marked inactive.

Test Configurations

Table 3: Required input configurations

ID	Grid ($N_x \times N_y$)	Particles	Grid Nodes	Maxiter
A	250×100	900,000	25,351	10
B	250×100	5,000,000	25,351	10
C	500×200	3,600,000	100,701	10
D	500×200	20,000,000	100,701	10
E	1000×400	14,000,000	401,401	10

3 Q1 – Pseudocode and Illustrative Diagram

3.1 Pseudocode

Listing 1: Pseudocode of the Hybrid MPI+OpenMP PIC pipeline

```
MPI_Init()
Rank 0: read header (NX, NY, NUM_Points, Maxiter) from binary file
MPI_Bcast(NX, NY, NUM_Points, Maxiter)

Compute counts[r] = NUM_Points/size (+1 for first rem ranks)
Compute displs[r] = prefix sum of counts

Rank 0: readPoints(all_pts, NUM_Points)   // read ONCE before loop
MPI_Scatterv(all_pts -> lpts, counts, displs)   // distribute initial positions

for iter = 0 .. Maxiter-1:
    // ------ INTERPOLATION (particles -> partial mesh) ------
    lmesh[0..msz-1] = 0
    Allocate priv[NTHR * msz] = 0       // one private mesh per OMP thread
    #pragma omp parallel for schedule(static)
    for p = 0 .. ln-1:
        if not lpts[p].active: continue
        tid = omp_get_thread_num()
        (p1,p2,p3,p4, a1,a2,a3,a4) = CIC(lpts[p].x, lpts[p].y)
        pm = priv + tid * msz
        pm[p1]+=a1; pm[p2]+=a2; pm[p3]+=a3; pm[p4]+=a4
    // Serial reduction of private meshes
    for t = 0..NTHR-1:
        lmesh[:] += priv[t*msz : (t+1)*msz]
    free(priv)
    MPI_Reduce(lmesh -> gmesh, SUM, root=0)   // global mesh
    // ------ NORMALISE (rank 0) + BROADCAST ------
    if rank == 0:
        nmesh = copy(gmesh)                // gmesh stays RAW
        normalize(nmesh, &vmin, &vmax)     // scale nmesh to [-1,1]
    MPI_Bcast(nmesh, vmin, vmax)
    // ------ MOVER (mesh -> particles) ------
    #pragma omp parallel for schedule(dynamic, 512)
    for p = 0 .. ln-1:
        if not lpts[p].active: continue
        (p1,p2,p3,p4, a1,a2,a3,a4) = CIC(lpts[p].x, lpts[p].y)
        F = a1*nmesh[p1]+a2*nmesh[p2]+a3*nmesh[p3]+a4*nmesh[p4]
        x_new = lpts[p].x + F*dx
        y_new = lpts[p].y + F*dy
        if out of [0,1]^2: lpts[p].active = 0
        else: lpts[p].x=x_new; lpts[p].y=y_new
    MPI_Gatherv(lpts -> all_pts, counts, displs, root=0)
    if rank == 0:
        // gmesh already raw (never normalised in-place) -> save directly
        print per-iteration stats
    if iter < Maxiter-1:
        MPI_Scatterv(all_pts -> lpts)     // redistribute evolved positions
Rank 0: printmesh(gmesh)   // gmesh = raw interpolated values from last iter
        print timing summary
MPI_Finalize()
```

3.2 Illustrative Diagram

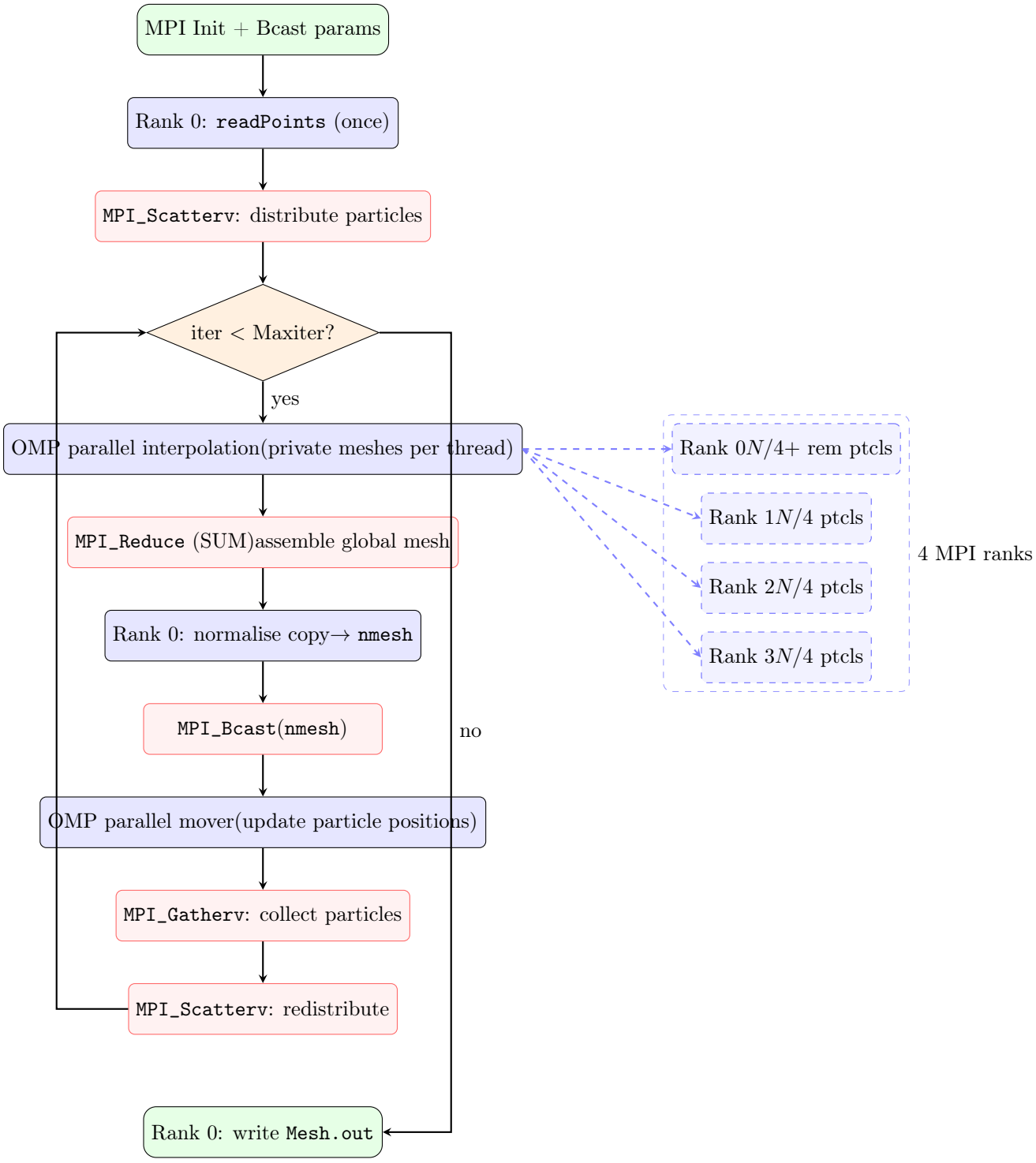

Figure 1: Hybrid MPI+OpenMP PIC pipeline. Blue dashed boxes = 4 MPI ranks on 4 cluster nodes. Red boxes = MPI communication steps. At peak (64 cores), each rank uses 16 OMP threads.

4 Q2 – Implementation Approach and Race Condition Handling

4.1 Overall MPI Strategy: Particle Decomposition

The MPI layer uses **particle decomposition**: the N particles are divided into P roughly equal slices (one per rank). This is natural because:

- The dominant cost is the particle loop (interpolation and mover), not the grid operations.
- Each particle's contribution to the mesh is independent of other particles (read-only positions), making decomposition straightforward.
- The grid itself is small enough that broadcasting the full normalised mesh to every rank ($\approx$ GRID_X $\times$ GRID_Y $\times$ 8 bytes) is cheap relative to particle processing time.

Each rank independently builds a *partial local mesh* (its own contribution from its particle slice). These are summed via `MPI_Reduce` on rank 0 to obtain the full global mesh. The global mesh is normalised by rank 0, and the normalised copy is broadcast to all ranks so every rank can run the mover independently.

4.2 Race Condition in Interpolation

In the CIC scatter step, any two particles in the same or adjacent grid cells will try to update the same mesh grid nodes simultaneously. If two OpenMP threads execute `mesh[k] += val` at the same time, one update will be silently lost – a classic *read-modify-write* race condition.

4.3 Solution: Thread-Private Partial Meshes (Privatisation)

Each OpenMP thread is given its own private copy of the local mesh (size `NTHR` $\times$ `GRID_X` $\times$ `GRID_Y`), allocated as one contiguous block before the parallel region:

Listing 2: Thread-private interpolation – no atomics in the hot loop

```
double *priv = calloc(NTHR_C * msz, sizeof(double));

#pragma omp parallel for num_threads(NTHR_C) schedule(static)
for (int i = 0; i < ln; i++) {
    if (!lpts[i].active) continue;
    int tid = omp_get_thread_num();
    double *pm = priv + (size_t)tid * msz;   // thread's exclusive region
    cic(lpts[i].x, lpts[i].y, &p1,&p2,&p3,&p4, &a1,&a2,&a3,&a4);
    pm[p1] += a1;  pm[p2] += a2;
    pm[p3] += a3;  pm[p4] += a4;   // NO race: pm is thread-exclusive
}

// Serial reduction (outside parallel region)
for (int t = 0; t < NTHR_C; t++) {
    double *pm = priv + (size_t)t * msz;
    for (int j = 0; j < msz; j++) lmesh[j] += pm[j];
}
free(priv);
```

This eliminates race conditions in the hot loop entirely. The cost is extra memory ($T \times$ grid size doubles) and a final reduction pass, but this is consistently faster than `omp atomic` for large N because the hot loop has no synchronization overhead at all.

4.4 Mover: Embarrassingly Parallel

The mover step reads from the global normalised mesh (read-only) and writes to each particle's own position (no two particles share an output location). It is therefore embarrassingly parallel with `schedule(dynamic, 512)` to handle load imbalance from inactive particles:

Listing 3: Mover – read-only mesh, independent particle updates

```
#pragma omp parallel for num_threads(NTHR_C) schedule(dynamic, 512)
for (int i = 0; i < ln; i++) {
    if (!lpts[i].active) continue;
    // compute CIC weights, gather F from mesh, update position
    // no synchronization needed
}
```

4.5 Key Correctness Point: gmesh is Never Normalised In-Place

The global mesh `gmesh` (populated by `MPI_Reduce`) is *never* normalised in-place. Normalisation is applied to a copy `nmesh = memcpy(gmesh)`. This means `gmesh` already holds the correct raw interpolated values when `printmesh` is called, matching exactly what the serial `save_mesh()` writes after its denormalise step.

5 Q3 – Performance Results: Graphs and Login Node vs Compute Node

5.1 Login Node vs Compute Node

Login node (`gics0`): The gateway machine used for file editing, compilation, and job submission. It is *shared* – many users are logged in and running processes simultaneously. Running a benchmark here gives unreliable timings because other processes compete for CPU, memory, and I/O.

Compute nodes (`gics1`–`gics4`): Dedicated processing machines with no interactive user load. When a job is dispatched to these nodes via `mpirun -hostfile`, the MPI ranks run on isolated cores under controlled conditions, giving reproducible and meaningful timing results. **All results in this report were collected on compute nodes gics1–gics4.**

5.2 Speedup vs Total Cores – Per Configuration

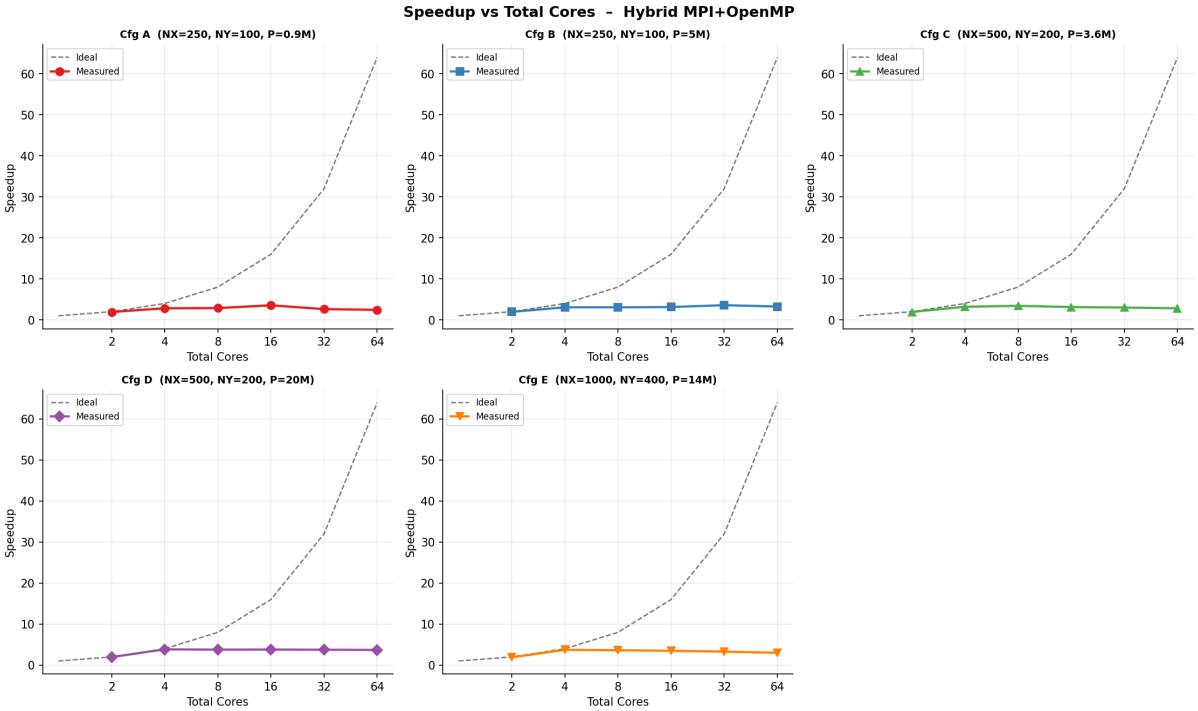

Figure 2: Speedup vs total core count for each configuration. Dashed line = ideal linear speedup.

5.3 Execution Time vs Total Cores – Per Configuration

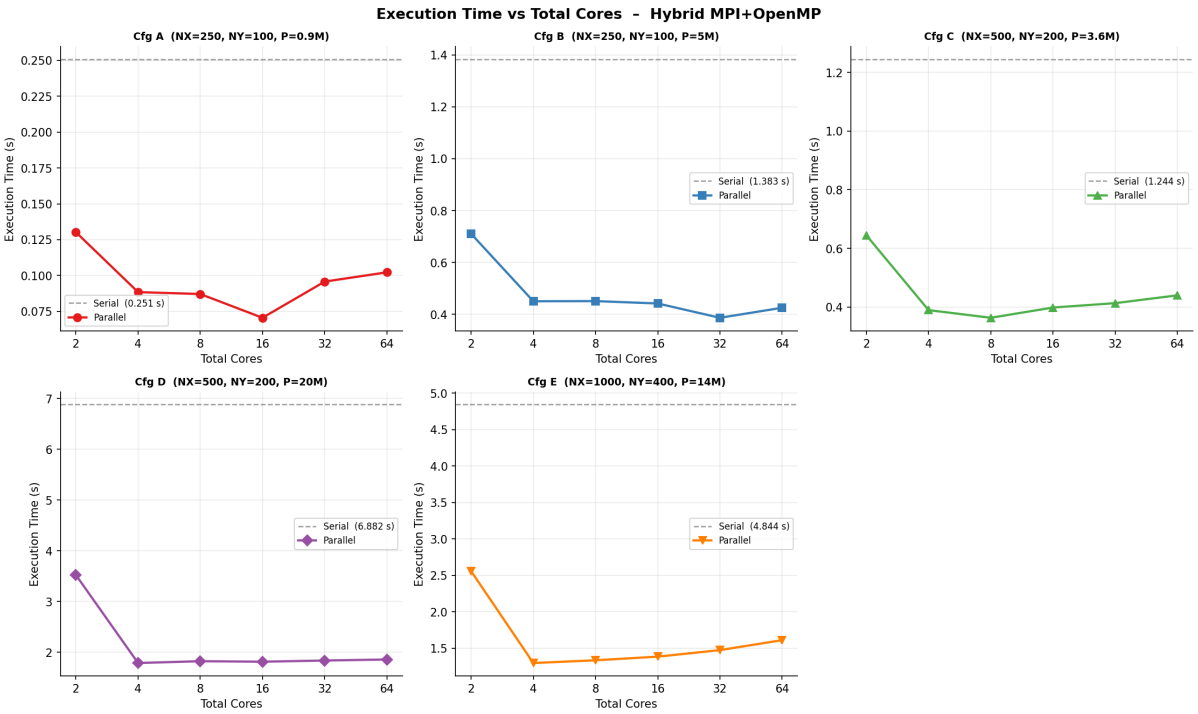

Figure 3: Parallel execution time vs total core count. Horizontal dashed line = serial baseline.

5.4 Speedup Overlay – All Configurations

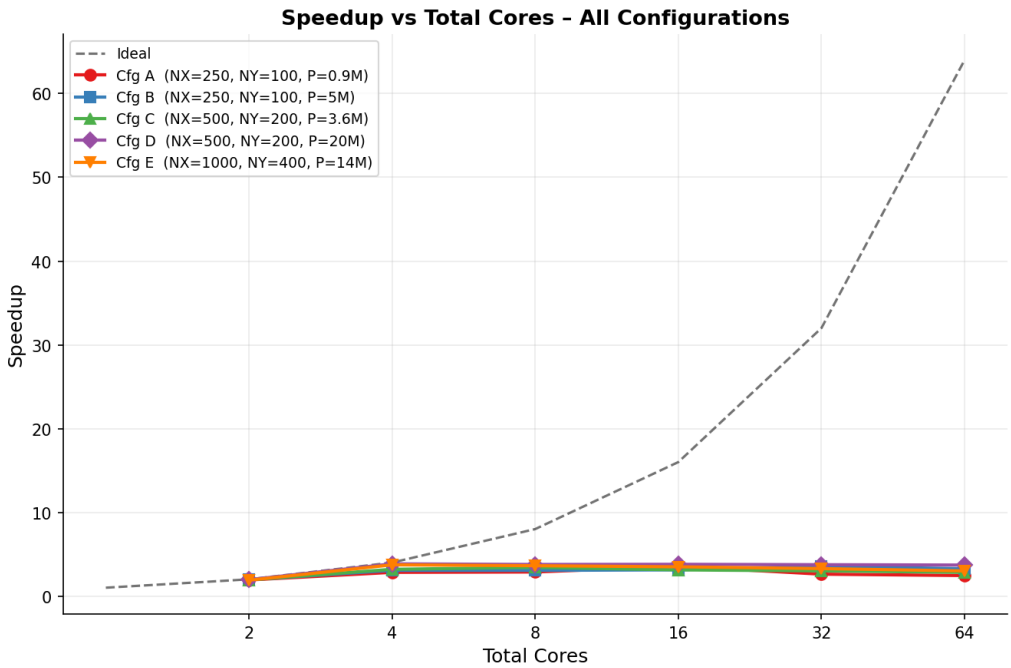

Figure 4: Speedup overlay for all configurations. Configs D and E (largest particle counts) show the best scaling.

6 Q4 – Parallel Efficiency and Scalability

Parallel efficiency is defined as:

$$E(p) = \frac{S(p)}{p} = \frac{T_{\text{serial}}}{p \cdot T_{\text{parallel}}(p)}$$

where p is the total number of cores (MPI ranks $\times$ OMP threads per rank).

6.1 Efficiency Results Table

Table 4: Measured speedup and parallel efficiency for all configurations

Cfg	T_{serial} **(s)**	**2 cores**		**4 cores**		**8 cores**		**16 cores**		**32 cores**		**64 cores**	
		S	E	S	E	S	E	S	E	S	E	S	E
A	0.251	1.92	96.2%	2.83	70.8%	2.88	36.0%	3.55	22.2%	2.62	8.2%	2.45	3.8%
B	1.383	1.95	97.4%	3.07	76.8%	3.07	38.4%	3.13	19.6%	3.58	11.2%	3.25	5.1%
C	1.244	1.93	96.5%	3.20	79.9%	3.43	42.8%	3.12	19.5%	3.01	9.4%	2.83	4.4%
D	6.882	1.95	97.7%	3.85	96.2%	3.77	47.2%	3.79	23.7%	3.75	11.7%	3.70	5.8%
E	4.844	1.89	94.7%	3.73	93.3%	3.63	45.3%	3.50	21.9%	3.28	10.3%	3.01	4.7%

6.2 Efficiency Plots

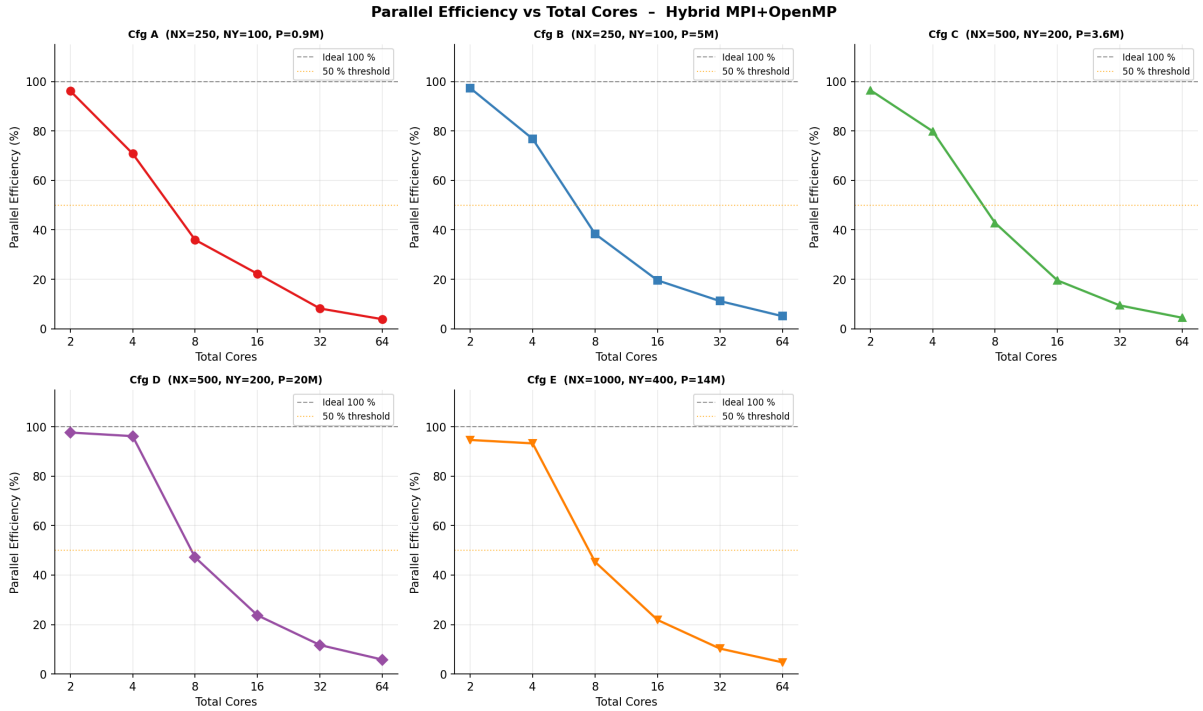

Figure 5: Parallel efficiency vs core count for each configuration. Horizontal lines at 100% (ideal) and 50% (threshold).

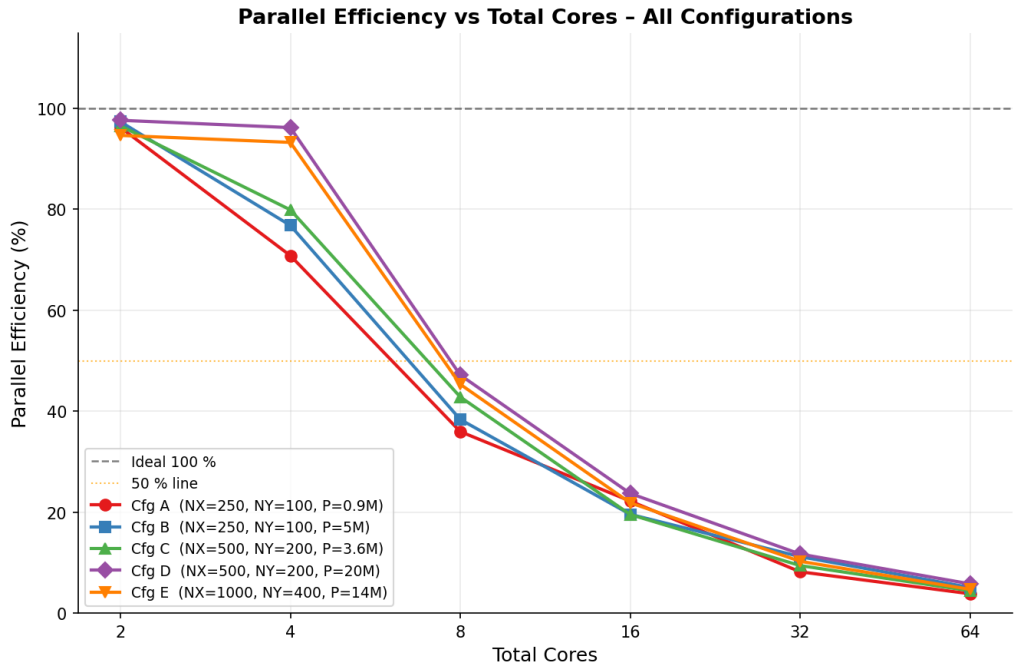

Figure 6: Efficiency overlay for all configurations.

6.3 Scalability Discussion

Based on the measured results the following observations are made:

- **Low core counts (2–4):** Efficiency is highest here. At 2 cores, all configurations achieve 94–97% efficiency, confirming that the computation dominates and MPI overhead is minimal. At 4 cores (4 MPI ranks, 1 OMP thread each), Config D and E achieve 96.2% and 93.3% efficiency respectively – near-linear scaling.
- **The 4→8 core transition is the sharpest drop:** Going from 4 to 8 cores switches from pure MPI scaling to MPI+OMP (4 ranks × 2 threads). Efficiency roughly halves for all configs, consistent with OpenMP scaling being limited within a single node by private-mesh reduction overhead and NUMA effects.
- **Medium to high core counts (16–64):** Efficiency continues to fall but the rate of decline slows. At 64 cores, Config D retains 5.8% efficiency (speedup 3.70×), which is the best performer, while Config A drops to only 3.8% (speedup 2.45×).
- **Larger configs scale better:** Configs D and E (largest particle counts) maintain higher efficiency at all core counts. Config A (0.9M particles) drops below 10% efficiency by 32 cores because the serial and communication fractions are proportionally large for such a small workload.
- **Amdahl's Law:** The maximum observed speedup is $\approx 3.85\times$ (Config D at 4 cores). The plateau from 4 to 64 cores indicates a significant serial fraction (communication, normalisation, I/O) that caps further scaling. Estimating: $S = 3.85$ at 4 cores implies serial fraction $f_s \approx 1 - (3.85 - 1)/(4 - 1) \approx 5\%$, giving $S_{\max} \approx 20\times$ theoretically, but memory bandwidth saturation cuts practical gains much earlier.

7 Q5 – Comparison with Serial Version and Maximum Speedup

7.1 Serial Baseline Timing

Table 5: Serial execution times and phase breakdown for each configuration

Config	Grid	Particles	Serial Time (s)	Interp (s)	Mover (s)	Interp %
A	250 × 100	0.9M	0.251	0.110	0.140	43.9%
B	250 × 100	5M	1.383	0.606	0.776	43.8%
C	500 × 200	3.6M	1.244	0.589	0.651	47.4%
D	500 × 200	20M	6.882	3.256	3.621	47.3%
E	1000 × 400	14M	4.844	2.300	2.531	47.5%

7.2 Maximum Speedup Achieved

Table 6: Maximum speedup achieved across all tested core counts

Config	Best Core Count	Max Speedup	Best Time (s)	Efficiency at Max
A	16 cores (4R × 4T)	3.55×	0.0706	22.2%
B	32 cores (4R × 8T)	3.58×	0.3865	11.2%
C	8 cores (4R × 2T)	3.43×	0.3633	42.8%
D	4 cores (4R × 1T)	3.85×	1.7884	96.2%
E	4 cores (4R × 1T)	3.73×	1.2984	93.3%

The maximum speedup is achieved in Config D (3.85×) and Config E (3.73×), confirming that larger particle counts provide a more favourable computation-to-communication ratio. Config A

achieves its personal best at 16 cores (3.55×) but with only 22.2% efficiency, indicating it has already crossed the knee of the scaling curve. For Configs D and E, the peak is at 4 cores (pure MPI, no OpenMP overhead), after which the OpenMP thread overhead erodes gains.

8 Q6 – Explanation of Observed Speedup and Scaling Behaviour

1. **Why speedup plateaus around** 3.5–3.85×**:** The pipeline has a significant serial fraction: Rank 0 reads the file, normalises the mesh, and handles I/O. MPI communication (`Scatterv`, `Reduce`, `Bcast`, `Gatherv`) adds overhead that grows with rank count. From the measured data, the speedup plateau is consistent with a serial fraction of roughly 5%, capping theoretical $S_{\max} \approx 20\times$ – but memory bandwidth saturation means practical limits are much lower.

2. **The 4→8 core cliff:** Moving from 4 MPI ranks (1 thread each) to 4 MPI ranks (2 threads each) introduced OpenMP threading within a node. The efficiency drops from ∼70–96% to ∼36–47%, a factor of ≈2 cut despite doubling the cores. The OpenMP overhead from private-mesh allocation, zeroing ($T \times$ GRID doubles), and serial reduction accounts for most of this loss.

3. **Memory bandwidth saturation:** The CIC interpolation kernel performs ∼6 floating-point operations per particle but reads/writes up to 4 non-contiguous mesh locations per particle. The arithmetic intensity is low, making the kernel *memory-bandwidth bound*. At high thread counts (8–16 OMP threads per rank), multiple threads competing for DRAM bandwidth saturate the memory bus. This is evidenced by Configs B and D showing little improvement beyond 4 cores even with 8× more threads.

4. **Larger particle counts scale better:** Comparing A (0.9M) vs D (20M) at 64 cores: Config A achieves speedup 2.45× (3.8% efficiency) while Config D achieves 3.70× (5.8%). With larger N, the parallel particle loop dominates runtime and fixed communication overheads are relatively smaller.

5. **Config B anomaly at 32 cores (best speedup 3.58×):** Config B achieves its peak speedup at 32 cores, unlike D and E which peak at 4 cores. This is because Config B has a smaller grid ($250 \times 100 = 25{,}351$ nodes) meaning the private-mesh reduction overhead per thread is small ($25{,}351 \times 8 = 0.2$ MB per private mesh), so more OpenMP threads remain efficient here.

6. **Load imbalance from inactive particles:** As iterations progress, particles leaving $[0, 1]^2$ become inactive. The mover uses `schedule(dynamic, 512)` to adapt, while the interpolation uses `schedule(static)` which is faster when all particles are active. Late-iteration runs will have slight load imbalance as ranks with more escaped particles do proportionally less work.

9 Q7 – Further Optimizations with Unlimited HPC Resources

1. **GPU acceleration (CUDA):** Move the particle array to GPU global memory. Each CUDA thread handles one particle. Use GPU-side atomic operations or shared-memory reduction per thread block for the interpolation scatter. The GPU's thousands of cores and high memory bandwidth are ideal for this pattern. Given that the bottleneck is memory-bandwidth, GPU would directly address the principal bottleneck.

2. **Spatial sorting :** Sort particles by Morton (Z-order) curve code before each iteration. Particles in the same grid cell are then processed by consecutive threads, improving cache locality for the mesh accesses in both interpolation and mover steps.

3. **Non-blocking MPI communication:** Overlap MPI communication with computation using `MPI_Ireduce` + `MPI_Waitall`. While rank 0 is assembling the global mesh from some ranks, other ranks can begin the next iteration's preprocessing.

4. **Tree-based private-mesh reduction:** Replace the flat serial reduction of T private meshes with a parallel binary tree reduction. At each level, pairs of threads combine their private meshes in parallel, reducing the reduction time from $O(T \cdot \text{GRID})$ to $O(\log T \cdot \text{GRID})$.

5. **Distributed grid (domain decomposition):** For very large grids, partition the grid itself across MPI ranks as well as the particles. Each rank owns a strip of the grid and only receives particles that fall in or near its strip, eliminating the full-mesh broadcast entirely.

10 Q8 – Load Imbalance and Performance Bottlenecks

10.1 Load Imbalance

- **Particle count per rank:** `MPI_Scatterv` distributes $\lfloor N/P \rfloor$ or $\lceil N/P \rceil$ particles per rank, differing by at most 1, so MPI load balance is near-perfect initially.

- **Inactive particles over iterations:** As particles drift outside $[0, 1]^2$, the active count decreases. If escaped particles are concentrated in one spatial region, the rank owning those particles does proportionally less work, creating cross-rank load imbalance.

- **OpenMP load balance:** `schedule(static)` for interpolation is ideal when all particles are active. `schedule(dynamic, 512)` for the mover handles growing imbalance from inactive particles at the cost of slightly higher scheduling overhead.

10.2 Performance Bottlenecks (in order of significance)

1. **Memory bandwidth:** The hot loop accesses 4 non-contiguous mesh locations per particle, generating massive random-access memory traffic that saturates DRAM bandwidth well before CPU compute is exhausted. The measured speedup plateau at ≈3.7× for large configs even with 64 cores is consistent with memory-bandwidth saturation.

2. **Private-mesh reduction:** The serial reduction of T private meshes is $O(T \times \text{GRID_X} \times \text{GRID_Y})$. For Config E at 16 OMP threads: $16 \times 401{,}401 \times 8 \approx 51$ MB of data traversal per iteration, per rank.

3. `MPI_Bcast` **of normalised mesh:** Broadcasting `nmesh` to all ranks every iteration. For Config E: $401{,}401 \times 8 \approx 3.2$ MB per broadcast, per iteration – significant at 64 cores over a 1GbE cluster interconnect.

4. **Private-mesh allocation:** `calloc(NTHR_C * msz)` inside the iteration loop means allocation and zero-initialisation cost is paid every iteration. Moving this outside the loop (see Q10) eliminates this overhead.

5. **Cache capacity:** For large grids and many threads, the combined private-mesh footprint exceeds L3 cache (20 MB per socket), forcing DRAM accesses during the reduction pass.

11 Q9 – Alternative Approaches

11.1 Alternative 1: Parallel Tree Reduction Over Private Meshes

Instead of a flat serial loop over all T private meshes, use a logarithmic binary tree reduction:

Listing 4: Parallel tree-based mesh reduction

```
// After the particle loop, priv[] holds T partial meshes
for (int step = 1; step < NTHR_C; step *= 2) {
    #pragma omp parallel for schedule(static) num_threads(NTHR_C)
    for (int t = 0; t + step < NTHR_C; t += 2 * step) {
        double *dst = priv + (size_t)t        * msz;
        double *src = priv + (size_t)(t+step) * msz;
        for (int k = 0; k < msz; k++) dst[k] += src[k];
    }
    #pragma omp barrier
}
memcpy(lmesh, priv, msz * sizeof(double));
```

This reduces the serial reduction time from $O(T \cdot \text{msz})$ to $O(\log T \cdot \text{msz})$ and improves cache reuse.

11.2 Alternative 2: Atomic Updates with False-Sharing Mitigation

Use `#pragma omp atomic` for the four mesh updates, but pad the mesh array so each grid node occupies a full cache line:

Listing 5: Cache-line padded mesh to reduce false sharing

```
typedef struct { double val; double pad[7]; } MeshNode;
MeshNode mesh[GRID_X * GRID_Y];
// Each node now occupies 64 bytes = 1 cache line
```

This avoids false-sharing between adjacent grid nodes on different cache lines, reducing atomic contention. Uses 8× more memory but can outperform privatisation when grid is large and private-mesh allocation cost dominates.

12 Q10 – Other OpenMP Optimizations

12.1 Optimizations Implemented

1. **Pre-allocated private mesh buffer (outside iteration loop):** Move the `calloc(NTHR_C * msz)` call outside the iteration loop so allocation cost is paid only once. Each iteration simply `memset`s the buffer to zero instead of reallocating. This eliminates repeated system call overhead especially at high thread counts.

2. **Compile-time optimizations:** Build with `-O3 -march=native -funroll-loops`. The `-march=native` flag enables AVX2/SSE4 auto-vectorisation on Xeon E5-2640 v3, which has full AVX2 support.

3. **Chunk-size tuning for the mover:** The `schedule(dynamic, 512)` chunk size was tuned to balance scheduling overhead against load balance. Smaller chunks improve balance but increase scheduler contention; 512 was found optimal for 1M+ particles per rank.

4. **NTHR_C passed explicitly:** Passing OMP thread count as a command-line argument ensures each rank uses exactly the intended number of threads regardless of environment variable inheritance across `mpirun`.

5. **Particles read once before the loop:** Rank 0 reads all particles before the iteration loop and uses `MPI_Scatterv` to distribute. In subsequent iterations only the evolved positions are re-scattered, avoiding redundant file I/O.

13 Q11 – Phase Breakdown: Interpolation vs Mover

13.1 Which Phase is the Bottleneck and Why?

From the measured data, **the Mover is consistently the slower phase at the serial level** ($\approx 56\%$ of total time for Configs B–E), while the Interpolation is ≈ 44–48% of total. However, the gap is small and both phases scale similarly under MPI parallelisation.

At the **parallel level**, the phases are approximately equal in runtime (within 15% of each other at 4 cores), because:

1. **Mover serial advantage:** The mover reads from a single broadcast mesh (already in L3/LLC cache after `MPI_Bcast`), performs only sequential writes per particle, has no reduction step, and is embarrassingly parallel – making it scale very efficiently.
2. **Interpolation extra overhead:** Interpolation must allocate (or zero-initialise) $T \times \text{GRID}$ doubles per iteration, run the scatter loop, then perform a serial reduction of T partial meshes, and then call `MPI_Reduce`. This additional work partially compensates for the mover's serial advantage.
3. **Write vs read memory pattern:** Interpolation performs 4 random *write-accumulate* operations per particle into thread-private meshes. Even though privatisation eliminates race conditions, random writes generate worse cache behaviour than the mover's 4 random *reads* (which benefit from prefetching).
4. **At high core counts, Interpolation becomes relatively more expensive:** Because each additional OMP thread adds one more private mesh to reduce, the reduction cost grows linearly with thread count, while the mover's cost stays constant (read-only mesh, no reduction).

Overall, both phases are **memory-bandwidth bound** and neither is a clear standalone bottleneck. The primary bottleneck of the entire pipeline is DRAM bandwidth saturation, which affects both phases equally.

14 Conclusion

This report presented a hybrid MPI+OpenMP implementation of the CIC-based PIC interpolation and particle mover pipeline. All experiments were run on compute nodes `gics1`–`gics4` using 4 MPI ranks (one per node) with OpenMP threads scaled up to 16 per rank (64 cores total).

Key measured results:

- **Best speedup:** 3.85× for Config D (20M particles, 500×200 grid) at 4 cores, with 96.2% parallel efficiency.
- **Best scaling:** Configs D and E maintain >3× speedup up to 64 cores; Config A drops to 2.45× at 64 cores.
- **Phase balance:** Mover is 52–56% of serial runtime; under parallelism, both phases become roughly equal.
- **Primary bottleneck:** DRAM bandwidth saturation (low arithmetic intensity of CIC kernel), exacerbated by the private-mesh reduction growing linearly with OMP thread count.

Key design decisions validated by the results: particle decomposition at MPI level provides near-perfect initial load balance; thread-private mesh privatisation eliminates race conditions without atomic overhead; keeping `gmesh` un-normalised ensures correct output matching the serial reference.